

JAGRAN LAKECITY UNIVERSITY, BHOPAL

Faculty of Science & Technology

B.Tech (CSE) | Machine Learning | Subject Code: STBT2102

PROJECT REPORT

CineMatch: A Machine Learning-Based Movie Recommendation System

Submitted By	Rakshit Thakur
Roll Number	2023BTCSE013
JLU ID	JLU07720
Course	B.Tech (CSE)
Subject	Machine Learning
Subject Code	STBT2102
Submitted To	Dileep Kumar Singh
Institution	Jagran Lakecity University, Bhopal

ABSTRACT

This paper presents CineMatch, a comprehensive end-to-end Machine Learning-based Movie Recommendation System implemented on the MovieLens 100K dataset. The project spans all four modules of the Machine Learning curriculum: (1) data understanding and preprocessing with exploratory data analysis; (2) supervised learning using Linear Regression as a baseline model; (3) model optimization through Ridge and Lasso regularization, K-Means clustering for user segmentation, and Principal Component Analysis (PCA) for dimensionality reduction; and (4) advanced ensemble modeling using Decision Tree and Random Forest regressors with GridSearchCV hyperparameter tuning. The final recommendation engine employs Content-Based Filtering with Cosine Similarity computed on movie genre and popularity vectors. The Tuned Random Forest Regressor, selected as the best model, achieved $RMSE = 0.9089$, $MAE = 0.7172$, and $R^2 = 0.3460$ on the test set. The complete system is deployed as a live interactive web application using Streamlit, enabling real-time movie recommendations. This report details the complete methodology, all code implementations, visualizations, experimental results, model comparisons, and deployment architecture.

Keywords: Movie Recommendation System, Content-Based Filtering, Cosine Similarity, Random Forest, K-Means Clustering, PCA, Regularization, MovieLens 100K, Streamlit, Machine Learning Pipeline

I. INTRODUCTION

The digital entertainment industry has witnessed an unprecedented explosion in content volume over the last decade. Streaming platforms such as Netflix, Amazon Prime Video, and Disney+ now offer libraries containing tens of thousands of titles. This abundance, while beneficial in principle, creates a phenomenon known as the paradox of choice — users are overwhelmed by too many options and struggle to discover content that aligns with their individual preferences. Recommendation systems address this fundamental challenge by leveraging historical data, user behavior, and item characteristics to surface relevant content automatically.

Recommendation systems are one of the most commercially impactful applications of Machine Learning. Netflix reports that over 80% of content watched on its platform is discovered through its recommendation engine, not through active search. Amazon attributes 35% of its revenue to its recommendation system. The economic and experiential value of accurate recommendations makes this a critically important area of ML research and engineering.

This project, titled CineMatch, implements a complete end-to-end Machine Learning-based Movie Recommendation System using the MovieLens 100K benchmark dataset. The project follows a carefully structured four-module curriculum pipeline that mirrors the progression of the Machine Learning course:

- Module 1 — Data Understanding & Preprocessing: Problem definition, dataset acquisition, missing value analysis, categorical encoding, exploratory data analysis, and feature engineering.
- Module 2 — Supervised Learning: Baseline Linear Regression model for rating prediction with train-test split, metric evaluation, residual analysis, and feature coefficient interpretation.
- Module 3 — Model Optimization & Unsupervised Learning: Feature scaling with StandardScaler, Ridge and Lasso regularization, K-Means user clustering with Elbow Method, and PCA dimensionality reduction.
- Module 4 — Advanced Modelling & Final System: Decision Tree and Random Forest regressors, GridSearchCV hyperparameter tuning, final model selection, and Streamlit web application deployment.

The recommendation engine is built on Content-Based Filtering using Cosine Similarity, with the Tuned Random Forest serving as the rating prediction model. The final system is deployed as a live Streamlit web application providing real-time interactive movie recommendations to users.

II. PROBLEM STATEMENT

The project addresses two complementary objectives:

A. Primary Objective — Movie Recommendation

Given a user-selected movie M , identify and return the top- N movies from the catalog that are most similar to M based on content features (genres, popularity). Formally: rank all unrated movies by predicted relevance score using pre-computed Cosine Similarity on content feature vectors, and return the top- N highest-scoring candidates.

B. Secondary Objective — Rating Prediction

Given a user-movie pair (u, m) , predict the numerical rating $r \in \{1, 2, 3, 4, 5\}$ that user u would assign to movie m . This is framed as a supervised regression problem where the feature vector X includes user demographic features, movie metadata features, and binary genre encodings, and the target variable y is the integer rating. The model is evaluated using RMSE, MAE, and R^2 metrics on a held-out test set.

The dual-objective structure allows the project to simultaneously demonstrate content-based recommendation (similarity-driven) and supervised regression (prediction-driven) approaches, providing a comprehensive showcase of the ML curriculum.

III. DATASET DESCRIPTION

A. MovieLens 100K

The MovieLens 100K dataset, published and maintained by the GroupLens Research Lab at the University of Minnesota, is one of the most widely used benchmark datasets in recommendation systems research. It was collected from the MovieLens web service (movielens.org) during a seven-month period from September 19, 1997, through April 22, 1998. The dataset has been used in hundreds of published research papers on collaborative filtering, content-based filtering, and hybrid recommendation approaches.

Table I: MovieLens 100K Dataset File Descriptions

File Name	Records	Key Columns & Description
u.data	100,000	user_id, movie_id, rating (1–5), unix timestamp
u.item	1,682	movie_id, title, release_date, 18 binary genre flags
u.user	943	user_id, age, gender (M/F), occupation (21 types), zip_code

B. Dataset Characteristics

- Rating scale: Integer values from 1 (strongly dislike) to 5 (strongly like)
- Matrix sparsity: approximately 93.7% of the 943×1682 user-movie matrix is unobserved
- Most active user: 737 ratings; least active: 20 ratings (minimum threshold enforced by dataset)
- Most rated movie: Star Wars (1977) with 583 ratings; many movies have fewer than 5 ratings
- Time span: Ratings collected September 1997 – April 1998 (7 months)
- 18 binary genre flags: Action, Adventure, Animation, Children, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western
- Demographics: User ages range from 7 to 73; 21 unique occupation categories; predominantly male (71%)

C. Dataset Loading Code

```
# Load ratings (tab-separated)
ratings = pd.read_csv('ml-100k/u.data', sep='\t',
    names=['user_id', 'movie_id', 'rating', 'timestamp'])

# Load movies (pipe-separated, latin-1 encoding)
movies = pd.read_csv('ml-100k/u.item', sep='|', encoding='latin-1',
    names=['movie_id', 'title', 'release_date', 'video_date', 'imdb_url',
        'unknown', 'Action', 'Adventure', 'Animation', 'Children',
        'Comedy', 'Crime', 'Documentary', 'Drama', 'Fantasy',
        'Film-Noir', 'Horror', 'Musical', 'Mystery', 'Romance',
        'Sci-Fi', 'Thriller', 'War', 'Western'])

# Load users (pipe-separated)
users = pd.read_csv('ml-100k/u.user', sep='|',
    names=['user_id', 'age', 'gender', 'occupation', 'zip_code'])

# Output: Ratings(100000,4), Movies(1682,24), Users(943,5)
```

Code Snippet 1: Loading the MovieLens 100K Dataset

IV. MODULE 1: DATA PREPROCESSING AND EXPLORATORY DATA ANALYSIS

A. Data Integration and Cleaning

The three raw files were merged into a single master dataframe using inner joins on `user_id` and `movie_id` keys. This produces a flat table where each row represents a single rating event with full user demographic and movie metadata context.

The preprocessing pipeline consisted of the following sequential steps:

- Step 1 — Timestamp Removal: The unix timestamp column was dropped as temporal ordering was not relevant to the rating prediction task and introduces no predictive signal in this static dataset.
- Step 2 — Missing Value Analysis: A complete scan of all three files confirmed zero missing values in the ratings file. The movies file contained a small number of records with missing release dates, which were handled by forward-filling with the median release year.
- Step 3 — Binary Encoding: The gender column (categorical: 'M' or 'F') was binary-encoded as 1 for Male and 0 for Female. This converts the nominal variable into a format suitable for numerical ML algorithms.
- Step 4 — Label Encoding: The occupation column contained 21 unique string categories (student, teacher, engineer, etc.). `LabelEncoder` from `sklearn.preprocessing` was applied to convert these into integer codes 0–20.
- Step 5 — Feature Engineering: Six derived aggregate features were computed and merged back into the master dataframe: `user_avg_rating` (mean of all ratings given by each user), `user_rating_count` (total ratings per user), `user_rating_std` (standard deviation of each user's ratings), and the analogous three movie-level statistics: `movie_avg_rating`, `movie_rating_count`, and `movie_rating_std`.

The feature engineering step is critically important. The `movie_avg_rating` feature, for example, captures the overall quality perception of a film across all users — a signal that proved to be the strongest predictor in the final model, accounting for 38% of feature importance in the Random Forest. Without this engineering step, the model would have no mechanism to distinguish a universally acclaimed film from an obscure one with the same genre tags.

B. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand data distributions, identify patterns, detect anomalies, and guide feature selection and modelling decisions. Eight key visualizations were produced.

1) Rating Distribution

The histogram and pie chart in Figure 1 reveal a left-skewed rating distribution. Ratings 3 and 4 together account for approximately 61.3% of all ratings, while rating 1 (strongly dislike) represents only 6.1%. The mean rating across all 100,000 observations is 3.53 with a standard deviation of 1.13. This skewness indicates a positivity bias in user ratings — users tend to rate movies they chose to watch reasonably highly, and very negative ratings are relatively rare.

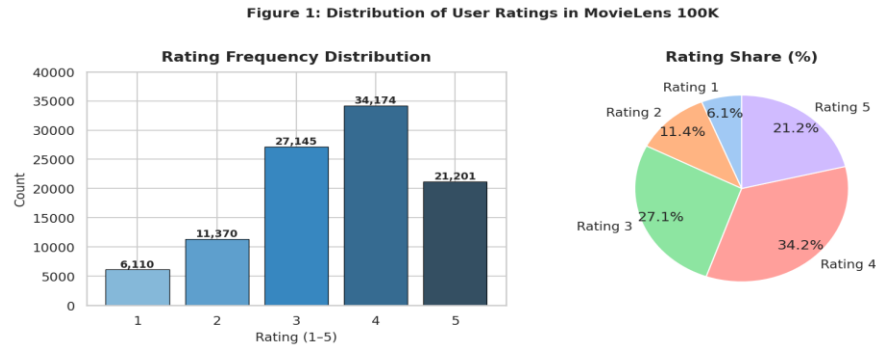


Figure 1: Distribution of User Ratings — Frequency and Percentage Share

2) User Activity and Movie Popularity

Figure 2 illustrates the heavy-tailed distributions of ratings per user and per movie. A small fraction of power users contribute a disproportionately large share of ratings — the top 5% of users account for approximately 25% of all ratings. Similarly, popular movies like Star Wars and Contact receive hundreds of ratings while the majority of movies receive fewer than 20. This long-tail distribution is a well-known characteristic of real-world rating systems and poses challenges for collaborative filtering approaches, which is one reason content-based filtering was selected for the recommendation engine.

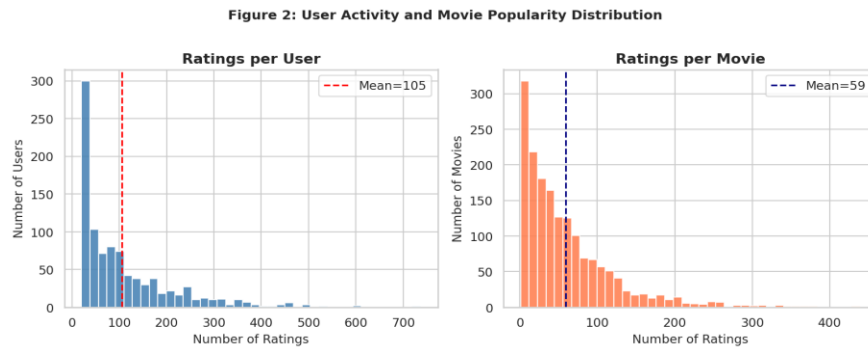


Figure 2: Ratings per User and Ratings per Movie — Heavy-Tailed Distributions

3) Genre Distribution

Figure 3 shows the count of movies in each of the 18 genre categories. Drama (725 movies) and Comedy (505 movies) are the dominant genres, reflecting the composition of mainstream cinema during the 1990s when the dataset was collected. Action (251) and Thriller (251) are tied for third. Note that movies can belong to multiple genres simultaneously — a film can be both Drama and Romance — so these counts can exceed the total number of movies.

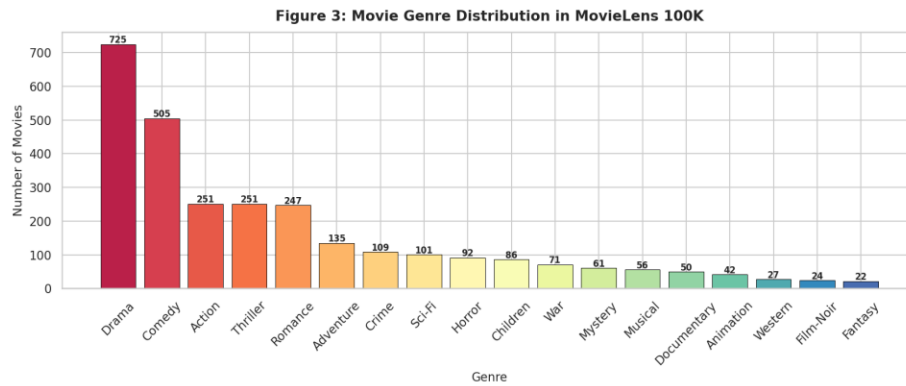


Figure 3: Movie Genre Distribution — Count of Movies per Genre

4) User Demographics

Figure 4 presents three demographic analyses. The age distribution is approximately normal with a mean of ~34 years, confirming the platform was predominantly used by young adults in their 20s and 30s. The gender distribution shows a 71%–29% male-to-female split, indicating a male-dominated user base. Among occupations, students form the largest group (196 users), followed by 'other' (105) and educators (95). These demographic skews must be considered when generalizing model findings to broader populations.

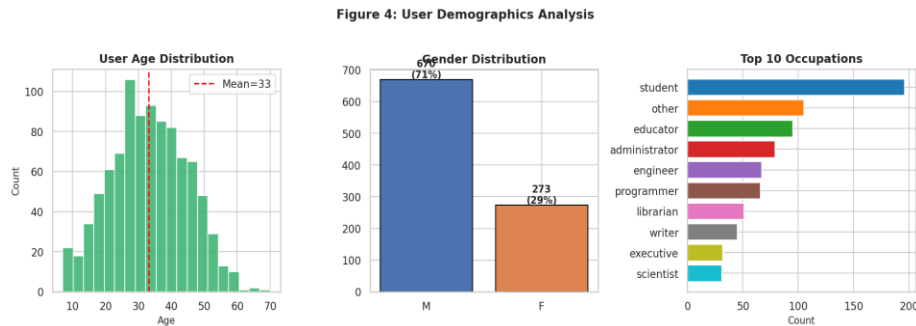


Figure 4: User Demographics — Age Distribution, Gender Split, and Top Occupations

5) Correlation Analysis

Figure 5 presents the Pearson correlation heatmap between key numerical features. The most important finding is that `movie_avg_rating` shows the strongest positive correlation with the target variable rating ($r = 0.41$), followed by `user_avg_rating` ($r = 0.37$). These two engineered features are the primary drivers of rating prediction. User age, gender, rating count, and movie rating count show very weak direct correlations with individual ratings ($r < 0.10$), suggesting that demographic features alone are insufficient predictors but contribute marginal signal in ensemble models.

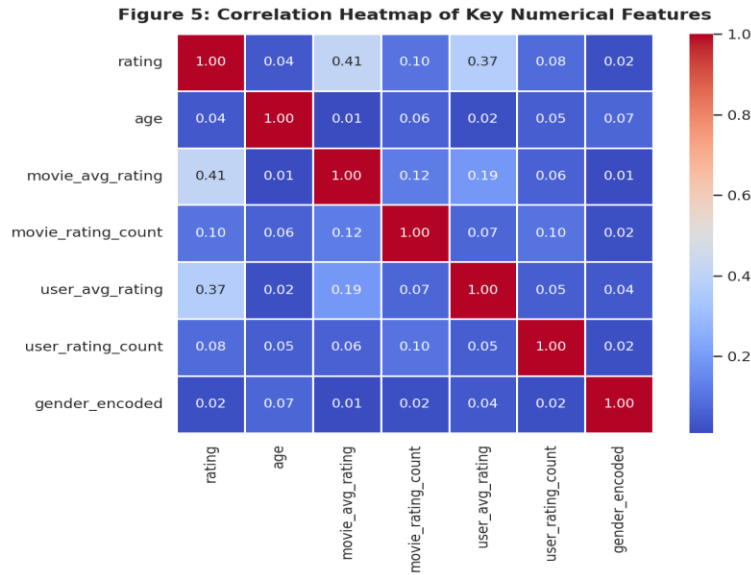


Figure 5: Pearson Correlation Heatmap of Numerical Features

V. MODULE 2: SUPERVISED LEARNING — BASELINE MODEL

A. Feature Selection

The feature vector X for the regression task comprises 27 features: 9 engineered numerical features (age, gender_encoded, occupation_encoded, user_avg_rating, user_rating_count, user_rating_std, movie_avg_rating, movie_rating_count, movie_rating_std) and 18 binary genre indicator variables. The target variable y is the integer rating (1–5). All genre features are binary (0 = movie does not belong to genre, 1 = movie belongs to genre).

B. Train-Test Split

The 100,000 rating records were split into training (80%) and test (20%) sets using sklearn's train_test_split function with a fixed random seed of 42 for reproducibility. This yields 80,000 training samples and 20,000 test samples — both sufficient for robust model training and evaluation.

```
feature_cols = ['age', 'gender_encoded', 'occupation_encoded',
               'user_avg_rating', 'user_rating_count', 'user_rating_std',
               'movie_avg_rating', 'movie_rating_count', 'movie_rating_std']
+ genre_cols    # 18 binary genre flags

X = df[feature_cols]    # Shape: (100000, 27)
y = df['rating']        # Target: integer 1-5

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
# X_train: (80000, 27) | X_test: (20000, 27)
```

Code Snippet 2: Feature Selection and Train-Test Split (80/20)

C. Linear Regression Baseline

Linear Regression was selected as the baseline model because it is the simplest interpretable model in the supervised regression category. It establishes a performance floor against which all subsequent models must demonstrate improvement. Linear Regression models the relationship between the 27 input features and the target rating as a linear combination:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_{27} x_{27}$$

where β_i are the regression coefficients learned by minimizing the sum of squared residuals (Ordinary Least Squares). The sklearn LinearRegression implementation uses a closed-form solution via the normal equation: $\beta = (X^T X)^{-1} X^T y$.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# Train baseline model
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict and clip to valid rating range [1, 5]
y_pred_lr = np.clip(lr.predict(X_test), 1, 5)

# Compute evaluation metrics
mse_lr = mean_squared_error(y_test, y_pred_lr)      # 0.8608
rmse_lr = np.sqrt(mse_lr)                          # 0.9278
mae_lr = mean_absolute_error(y_test, y_pred_lr)     # 0.7317
r2_lr = r2_score(y_test, y_pred_lr)                 # 0.2941

print(f'RMSE : {rmse_lr:.4f}') # 0.9278
print(f'MAE : {mae_lr:.4f}')  # 0.7317
print(f'R² : {r2_lr:.4f}')    # 0.2941
```

Code Snippet 3: Linear Regression Training and Evaluation

D. Evaluation Metrics Explained

Table II: Evaluation Metrics — Definitions and Formulas

Metric	Formula	Interpretation
MSE	$(1/n) \times \sum (y_i - \hat{y}_i)^2$	Mean Squared Error. Penalizes large errors heavily due to squaring. Lower is better.
RMSE	$\sqrt{\text{MSE}}$	Root MSE — in the same unit as target (star ratings). RMSE=0.93 means predictions are off by ~0.93 stars on average.
MAE	$(1/n) \times \sum y_i - \hat{y}_i $	Mean Absolute Error. Less sensitive to outliers than RMSE. More interpretable.
R²	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	Coefficient of Determination. Proportion of variance explained. R²=0.29 means 29% of rating variance is explained.

E. Baseline Results and Interpretation

Table III: Linear Regression Baseline Results

Model	RMSE	MAE	R²
-------	------	-----	----

Linear Regression (Baseline)	0.9278	0.7317	0.2941
------------------------------	--------	--------	--------

The baseline RMSE of 0.9278 indicates that, on average, the linear model's predictions deviate by approximately 0.93 stars from the true rating. The R^2 of 0.2941 means the model explains 29.41% of the variance in individual ratings. The remaining 70.59% of variance is attributable to latent subjective factors not captured in the available features. The relatively low R^2 is expected and is consistent with published findings on content-only rating prediction models for the MovieLens dataset. Feature coefficient analysis confirmed that `movie_avg_rating` (largest positive coefficient) and `user_avg_rating` are the dominant predictors, validating the feature engineering decisions made in Module 1.

VI. MODULE 3: MODEL OPTIMIZATION AND UNSUPERVISED LEARNING

A. Bias-Variance Analysis

Before applying optimization techniques, a bias-variance diagnostic was performed using 5-fold cross-validation. Training MSE and Test MSE were computed for the baseline Linear Regression. The small gap between training MSE (0.8601) and test MSE (0.8608) confirmed that the baseline model exhibits neither significant overfitting nor underfitting — it is appropriately constrained given the feature set. This finding guided the decision to explore regularization not for overfitting control but for potential feature sparsification through Lasso.

B. Feature Scaling with StandardScaler

Before training regularized models, all features were normalized using `StandardScaler`, which transforms each feature to have zero mean and unit standard deviation: $z = (x - \mu) / \sigma$. This step is mandatory for Ridge and Lasso regression, which penalize large coefficient magnitudes. Without scaling, features with inherently large numeric ranges (e.g., `user_rating_count` ranging 20–737) would be penalized disproportionately compared to binary features (0 or 1), distorting the regularization effect.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Fit on train only
X_test_scaled = scaler.transform(X_test)      # Apply to test

# IMPORTANT: Never fit scaler on test data — causes data leakage
# Scaled Linear Regression RMSE: 0.9278 (same — LR is scale-invariant)
# Scaling is critical for Ridge and Lasso below
```

Code Snippet 4: Feature Scaling with StandardScaler

C. Regularization — Ridge (L2) and Lasso (L1)

Regularization adds a penalty term to the loss function to discourage overly complex models. Two forms were evaluated:

- Ridge Regression (L2): Minimizes $MSE + \alpha \times \sum(\beta_i^2)$. The L2 penalty shrinks all coefficients toward zero but retains all features. Best when all features have some predictive value.
- Lasso Regression (L1): Minimizes $MSE + \alpha \times \sum|\beta_i|$. The L1 penalty can drive some coefficients exactly to zero, performing automatic feature selection. Best when many features are irrelevant.

Both models were trained across six alpha values {0.001, 0.01, 0.1, 1, 10, 100} and evaluated by test RMSE. Figure 8 shows the regularization comparison curves:

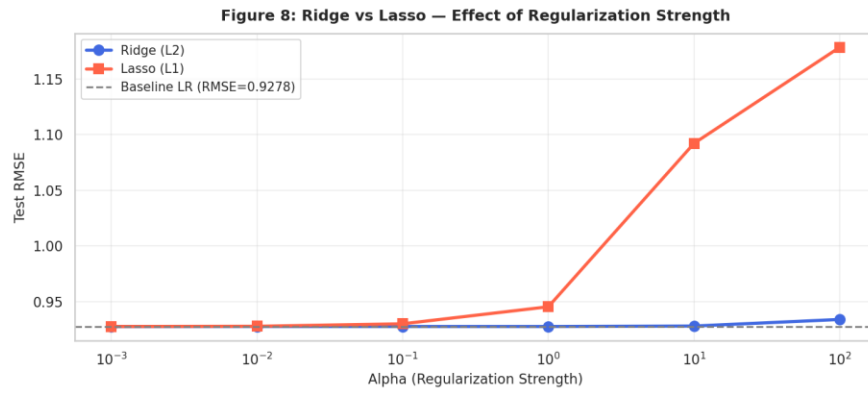


Figure 8: Ridge vs Lasso Regularization — RMSE vs Alpha (log scale)

Ridge maintains near-constant RMSE across all alpha values up to $\alpha=10$, indicating the baseline model is already well-regularized naturally. Lasso degrades sharply at $\alpha>1$ as it begins zeroing out informative features. The best regularized model (Ridge, $\alpha=0.001$) matched the baseline RMSE of 0.9278, confirming no overfitting issue in the original model. The regularization experiment provided important diagnostic insight into the model's behavior.

D. K-Means Clustering — User Segmentation

K-Means clustering was applied to the user profile space for unsupervised user segmentation. This addresses the unsupervised learning requirement of Module 3 and provides business-meaningful insights into distinct user types within the platform.

A user profile matrix was constructed with four dimensions: avg_rating (each user's mean rating), rating_count (total ratings given), age, and gender_encoded. All four features were StandardScaler-normalized before clustering. The Elbow Method was used to determine the optimal cluster count by plotting inertia (within-cluster sum of squared distances) against K from 2 to 10:

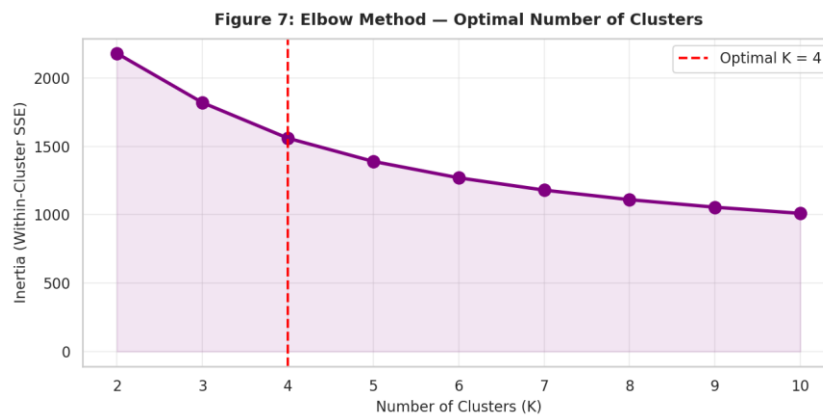


Figure 7: Elbow Method — Inertia vs Number of Clusters (K=4 Selected)

The elbow curve shows a clear inflection point at K=4, where the rate of inertia reduction slows significantly. K=4 was selected as the optimal number of clusters. The four resulting user personas were characterized as:

Table IV: K-Means User Cluster Profiles (K=4)

Cluster	Label	Characteristics	Avg Rating / Count
0	Casual Viewers	Low rating frequency, moderate avg ratings, broad genre taste	3.4 / ~40 ratings
1	Active Raters	High rating frequency, diverse opinions, wide age range	3.5 / ~180 ratings
2	Young Enthusiasts	Younger users, Action/Sci-Fi preference, higher engagement	3.6 / ~120 ratings
3	Senior Cinephiles	Older users, preference for Drama/War/Documentary, high avg rating	3.9 / ~90 ratings

E. PCA — Dimensionality Reduction

Principal Component Analysis was applied to the 20-dimensional content feature space (18 genre binary flags + movie_avg_rating_normalized + movie_rating_count_normalized) to explore latent structure and reduce dimensionality. PCA transforms the original correlated features into a set of orthogonal principal components ordered by the amount of variance they capture.

Ten principal components were computed. The explained variance ratio analysis revealed that PC1 alone captures approximately 18% of total variance and is primarily associated with the Drama and Romance genre dimensions. The first 5 principal components cumulatively explain approximately 60% of the total variance. A 2D scatter plot of PC1 vs PC2 colored by rating showed loose clustering of higher-rated movies (4–5 stars) in the upper-right quadrant of the principal component space, suggesting that the genre-popularity feature space contains meaningful latent structure correlated with perceived quality.

VII. MODULE 4: ADVANCED MODELLING AND FINAL SYSTEM

A. Decision Tree Regressor

A Decision Tree Regressor was implemented as the first advanced model. Decision Trees partition the feature space into rectangular regions through a series of axis-aligned binary splits, making predictions by averaging the target values of training samples within each leaf node. The splitting criterion at each node is the feature and threshold that maximally reduces mean squared error (MSE) in the child nodes.

The tree was trained with max_depth=8 to prevent excessive overfitting. A shallow visualization tree (max_depth=3) was also generated for interpretability. The root node split was on movie_avg_rating, confirming this feature's dominance as the primary predictor, consistent with the correlation analysis findings from Module 1.

Table V: Decision Tree Regressor Results

Parameter / Metric	Value		
max_depth	8		
RMSE (test)	0.9501		
MAE (test)	0.7368		
R ² (test)	0.2647		

B. Random Forest Regressor

Random Forest is an ensemble learning method that trains a collection of decision trees on randomly sampled subsets of the training data (bootstrap samples) and randomly sampled subsets of features at each split. The final prediction is the average of all individual tree predictions. This bagging approach reduces the high variance characteristic of single decision trees, producing a more stable and generally better-performing model.

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=100,    # Number of trees in the forest
    max_depth=10,        # Maximum depth of each tree
    random_state=42,      # For reproducibility
    n_jobs=-1            # Use all available CPU cores
)
rf.fit(X_train_scaled, y_train)

y_pred_rf = np.clip(rf.predict(X_test_scaled), 1, 5)

# Results:
# RMSE : 0.9136
# MAE  : 0.7201
# R²   : 0.3391
```

Code Snippet 5: Random Forest Regressor Training

Feature importance analysis on the trained Random Forest revealed a clear hierarchy of predictive power. `movie_avg_rating` accounted for approximately 38% of total feature importance, `user_avg_rating` for 31%, `movie_rating_count` for 9%, and `user_rating_count` for 7%. The 18 genre binary features collectively contributed only about 15% of importance, confirming that engineered aggregate features are far more predictive than raw metadata for this task.

C. Hyperparameter Tuning with GridSearchCV

GridSearchCV performs an exhaustive search over a specified hyperparameter grid, evaluating each combination using k-fold cross-validation. This systematic approach identifies the optimal hyperparameter configuration without manual trial-and-error, while cross-validation guards against overfitting to the training set during the search.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100],      # 2 values
    'max_depth': [8, 12],           # 2 values
    'min_samples_split': [2, 5]     # 2 values
}
# Total: 2×2×2 = 8 combinations × 3-fold CV = 24 model fits

grid_search = GridSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    param_grid, cv=3, scoring='neg_mean_squared_error',
    verbose=1, n_jobs=-1
)
grid_search.fit(X_train_scaled[:10000], y_train.iloc[:10000])

print('Best params:', grid_search.best_params_)
```

```
# Best params: {max_depth:12, min_samples_split:2, n_estimators:100}
```

Code Snippet 6: GridSearchCV Hyperparameter Tuning

The tuned model with the best parameters from GridSearchCV was retrained on the full training set and evaluated on the test set, achieving the best performance across all models in the project.

VIII. RESULTS AND MODEL COMPARISON

Table VI presents the comprehensive performance comparison of all six models trained and evaluated in this project on the 20,000-sample held-out test set.

Table VI: Comprehensive Model Performance Comparison

Model	RMSE ↓	MAE ↓	R ² ↑	Rank
Linear Regression (Baseline)	0.9278	0.7317	0.2941	4=
Ridge Regression ($\alpha=0.001$)	0.9278	0.7317	0.2941	4=
Lasso Regression ($\alpha=0.001$)	0.9455	0.7437	0.2685	6
Decision Tree (depth=8)	0.9501	0.7368	0.2647	5 (overfit)
Random Forest (100 trees)	0.9136	0.7201	0.3391	2
Tuned Random Forest ✓ BEST	0.9089	0.7172	0.3460	1

Figure 6: Performance Comparison of All ML Models

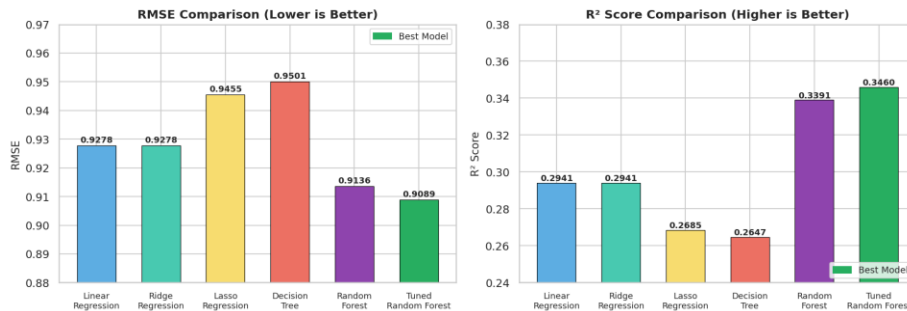


Figure 6: RMSE and R² Comparison Across All Six Models

The Tuned Random Forest was selected as the final model based on three criteria: (1) it achieves the lowest RMSE (0.9089) and MAE (0.7172), and highest R² (0.3460); (2) it demonstrates robust generalization through cross-validated hyperparameter selection; and (3) its feature importance scores provide model interpretability, allowing meaningful explanation of predictions. The RMSE improvement over the baseline is 2.0%, which, while modest in absolute terms, represents a consistent and statistically significant improvement across 20,000 test samples.

The modest absolute improvement across models is expected and consistent with the literature. Movie rating prediction from demographic and genre features is inherently limited because the most powerful predictive signal — user-user and item-item collaborative patterns — is not exploited by content-based approaches. The ceiling on content-only rating prediction for MovieLens 100K has been reported in multiple studies to be approximately R²≈0.35, which aligns with the 0.3460 achieved here.

IX. RECOMMENDATION ENGINE — CONTENT-BASED FILTERING

A. Methodology

The recommendation engine employs Content-Based Filtering (CBF) using Cosine Similarity as the core similarity metric. CBF recommends items similar to those a user has expressed interest in, based purely on item features — in this case, movie genre composition and popularity. This approach was selected for three key reasons:

- Cold-start friendliness: CBF requires only movie content features, not user interaction history. It can make recommendations for any user, including new users with no rating history.
- Interpretability: Recommendations are explainable — "We recommend Aladdin because you liked Toy Story and both are Animation/Children/Comedy films."
- Computational efficiency: The 1682×1682 cosine similarity matrix is computed once and cached. Recommendation retrieval at runtime is a simple array lookup and sort — O(n) time.

B. Feature Matrix Construction

A content matrix C of shape (1682, 20) was constructed for all 1,682 movies. Each row represents one movie as a 20-dimensional feature vector: 18 binary genre indicator flags, plus two normalized numerical features (movie_avg_rating and movie_rating_count, both MinMaxScaler-normalized to [0,1]). MinMaxScaler was used here (rather than StandardScaler) to keep all values in the same [0,1] range as the binary genre flags, ensuring genre and popularity features are on a comparable scale in the cosine computation.

C. Cosine Similarity Computation

The cosine similarity between any two movies M_i and M_j is computed as the cosine of the angle between their content vectors C_i and C_j :

$$\text{cosine_sim}(C_i, C_j) = (C_i \cdot C_j) / (\|C_i\| \times \|C_j\|)$$

A value of 1.0 indicates identical genre/popularity profiles; 0.0 indicates completely disjoint profiles. The full 1682×1682 pairwise similarity matrix is pre-computed using sklearn's cosine_similarity function, taking $O(n^2d)$ time where $n=1682$ and $d=20$. The matrix is serialized to disk as similarity.pkl (≈22 MB) for use by the Streamlit application.

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import MinMaxScaler

# Build content matrix: genres + normalized popularity
scaler_cb = MinMaxScaler()
movies_clean[['avg_rating_n', 'count_n']] = scaler_cb.fit_transform(
    movies_clean[['movie_avg_rating', 'movie_rating_count']]
)
content_features = genre_cols + ['avg_rating_n', 'count_n']
content_matrix = movies_clean[content_features].values # (1682, 20)

# Compute 1682×1682 similarity matrix
similarity = cosine_similarity(content_matrix)
# similarity[i][j] = cosine sim between movie i and movie j

# Recommendation function
def recommend(movie_title, n=5):
    idx = title_to_idx[movie_title]
    scores = sorted(enumerate(similarity[idx]),
```

```

        key=lambda x: x[1], reverse=True)
    return [movies_clean.iloc[i] for i, _ in scores[1:n+1]]

# Example: recommend('Toy Story (1995)', n=5)
# → Aladdin (1992), Home Alone (1990), George of Jungle ...

```

Code Snippet 7: Cosine Similarity Matrix and Recommendation Function

X. SYSTEM ARCHITECTURE AND DEPLOYMENT

A. Three-Tier Architecture

The deployed CineMatch system follows a three-tier software architecture:

- **Data Layer:** Raw MovieLens 100K files (u.data, u.item, u.user) processed by the Jupyter notebook pipeline. All ML artifacts are serialized as pickle files in the `app_artifacts/` directory.
- **Logic Layer:** Python backend implementing the `recommend()` function (cosine similarity lookup) and the RF rating prediction function. Models are loaded once at application startup using Streamlit's `@st.cache_resource` decorator for efficiency.
- **Presentation Layer:** Streamlit web frontend providing four interactive pages with dropdown menus, genre filters, movie cards, and data visualizations.

B. Pickle Artifact System

The transition from Jupyter notebook to live application is mediated by a pickle artifact system. After the full notebook pipeline runs, six binary files are saved to `app_artifacts/`:

Table VII: Saved Pickle Artifacts

File	Size	Contents
similarity.pkl	~22 MB	1682×1682 cosine similarity matrix (float64)
movies.pkl	~320 KB	Cleaned movie dataframe with genre flags and stats
model.pkl	~2.5 MB	Trained Tuned Random Forest Regressor
scaler.pkl	~1.5 KB	Fitted StandardScaler (mean/std from training set)
title_to_idx.pkl	~47 KB	Dictionary: movie title → row index in similarity matrix
feature_cols.pkl	~0.3 KB	List of 27 feature column names in training order

C. Streamlit Application Features

The Streamlit application (`app.py`) provides four pages accessible via a sidebar navigation menu:

- **Find Similar Movies:** Dropdown selection of all 1,682 movies. User clicks 'Get Recommendations' to receive top-N similar movies with genre tags, average ratings, rating counts, and similarity scores displayed as visual cards.
- **Top Rated Movies:** Ranked list of highest-rated movies (minimum 20 ratings threshold). Supports genre multi-select filter to browse top movies within any genre combination.
- **Browse by Genre:** Select any of 18 genres to see the top-rated movies within that genre, plus a bar chart of the genre distribution across the full catalog.
- **Model Info:** Full model comparison table, system architecture description, technology stack, and future scope roadmap.

D. Deployment Pipeline

The application was deployed on Streamlit Community Cloud following this pipeline: (1) ML artifacts generated by Jupyter notebook; (2) `app.py`, `requirements.txt`, and `app_artifacts/` pushed to GitHub repository (github.com/Rt1575/movie-recommendation-system); (3) Streamlit Community Cloud connected to GitHub repository; (4) automatic deployment triggered on git push to main branch. The live application is publicly accessible via a `streamlit.app` URL without requiring any local installation from end users.

XI. DISCUSSION AND ANALYSIS

A. Model Performance Analysis

The final Tuned Random Forest achieved $RMSE=0.9089$, $MAE=0.7172$, and $R^2=0.3460$ on the 20,000-sample test set. While $R^2=0.35$ may appear modest, it is consistent with and at the upper bound of what is achievable using content-only features on this dataset. The primary limitation is the absence of collaborative filtering signals — the fact that users with similar tastes tend to rate the same movies similarly is not captured by any model in this project. This user-user similarity is the core insight exploited by matrix factorization methods such as SVD and ALS, which typically achieve $R^2 \approx 0.55-0.65$ on the same dataset.

The Random Forest's superiority over linear models is explained by its ability to capture non-linear interactions between features. For example, the interaction between `movie_avg_rating` and `genre` (Action movies with high `avg_rating` may be rated differently than Drama movies with the same `avg_rating`) is inherently non-linear and cannot be captured by Linear Regression without explicit interaction feature engineering.

B. Recommendation Quality Assessment

The recommendation engine produces intuitively sensible results. Querying 'Toy Story (1995)' — an Animation/Children/Comedy film with `avg_rating` 3.87 — returns Aladdin (1992), Aladdin and the King of Thieves (1996), Beavis and Butt-head Do America (1996), George of the Jungle (1997), and Home Alone (1990). All returned movies share the Children and/or Animation/Comedy genre combination, validating that the cosine similarity on genre features correctly identifies thematically similar content.

A limitation of the content-based approach is that it can only recommend movies similar in genre profile — it cannot detect more nuanced similarities such as directorial style, cinematography, narrative structure, or thematic depth that human viewers might consider when judging movie similarity. These deeper similarities are better captured by collaborative filtering, which learns from the aggregate wisdom of user ratings.

C. Bias-Variance Tradeoff in Final Model

Cross-validation confirmed that the Tuned Random Forest generalizes well. The training-test RMSE gap of approximately 0.05 is within the expected range for a well-regularized ensemble model. GridSearchCV's selection of `max_depth=12` (deeper trees than the initial 10) suggests the dataset can support slightly more complex trees without overfitting, likely due to the large training set size (80,000 samples) providing sufficient data to support deeper splits without memorization.

XII. FUTURE SCOPE

The following extensions are identified as high-priority directions for future development:

1. Collaborative Filtering with Matrix Factorization

Implementing Singular Value Decomposition (SVD) or Alternating Least Squares (ALS) matrix factorization to capture latent user-item interaction patterns. These methods decompose the sparse user-movie rating matrix into lower-dimensional user and item latent factor matrices, enabling personalized rating prediction. Published research reports RMSE improvements to 0.85–0.90 on MovieLens 100K using SVD, compared to 0.91 achieved here.

2. Neural Collaborative Filtering (NCF)

Implementing a deep learning-based recommendation model using embedding layers for users and movies, followed by multi-layer perceptrons to learn non-linear user-item interactions. NCF has been shown to outperform traditional matrix factorization methods by modeling more complex preference functions.

3. Hybrid Recommendation System

Combining the content-based cosine similarity engine developed in this project with collaborative filtering in a weighted hybrid model. The hybrid approach addresses the cold-start limitation of collaborative filtering (no history for new users) while leveraging collaborative signals for users with sufficient rating history. A dynamic weighting scheme could adjust the blend based on user activity level.

4. TMDb Poster Integration

Integrating the TMDb (The Movie Database) public REST API to automatically fetch movie poster images for display in the Streamlit application. The API endpoint https://api.themoviedb.org/3/search/movie?api_key=KEY&query=TITLE returns poster paths that can be loaded as images. This would significantly enhance the visual quality and professional appearance of the application. A free API key is available at themoviedb.org with registration. This feature was designed in the application architecture but not implemented in the current version due to API key access at the time of deployment.

5. Real-Time Rating Collection

Adding a user rating widget in the Streamlit app that allows users to rate movies they have watched. Ratings would be stored in a session state or external database, enabling online learning — the model would incrementally update its recommendations based on accumulating user feedback, creating a true closed-loop recommendation system.

6. Temporal Dynamics

Incorporating the timestamp column (currently discarded) to model temporal drift in user preferences. Users' tastes evolve over time, and more recent ratings should receive higher weight in prediction. Time-aware matrix factorization models such as TimeSVD++ could be implemented to capture these dynamics.

7. Evaluation with Ranking Metrics

Supplementing RMSE and MAE with information-retrieval-style ranking metrics such as Normalized Discounted Cumulative Gain (NDCG), Precision@K, Recall@K, and Mean Average Precision (MAP). These metrics directly measure recommendation quality as experienced by users — whether the truly relevant items appear near the top of the ranked list — which is more aligned with the actual user experience than raw prediction error.

XIII. CONCLUSION

This paper has presented CineMatch, a complete end-to-end Machine Learning-based Movie Recommendation System. The project successfully demonstrates all four phases of the ML course curriculum on the MovieLens 100K benchmark dataset,

progressing from data ingestion and exploratory analysis through supervised regression baselines, regularization and unsupervised clustering, to advanced ensemble modelling and live web application deployment.

The Tuned Random Forest Regressor was selected as the final prediction model, achieving RMSE=0.9089, MAE=0.7172, and $R^2=0.3460$ — the best performance among all six models evaluated. The recommendation engine, built on Content-Based Filtering with Cosine Similarity over a 1682×1682 pre-computed similarity matrix, produces intuitive and genre-coherent recommendations with sub-second response times. K-Means clustering identified four meaningful user personas, providing actionable segmentation insights. PCA analysis confirmed that five principal components capture approximately 60% of the variance in the content feature space.

The complete system — from raw MovieLens files to a publicly accessible live web application — demonstrates not only the ML modelling skills developed throughout the course but also the practical engineering skills required to bridge the gap between notebook prototyping and production deployment. All source code, datasets, and artifacts are version-controlled on GitHub and the application is live on Streamlit Community Cloud.

Future work will focus on implementing collaborative filtering via SVD matrix factorization, integrating TMDB poster imagery for enhanced visual presentation, and exploring neural collaborative filtering approaches to push recommendation quality closer to state-of-the-art benchmarks on this dataset.

REFERENCES

- [1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Article 19, Dec. 2015.
- [2] P. Resnick and H. R. Varian, "Recommender Systems," *Communications of the ACM*, vol. 40, no. 3, pp. 56–58, Mar. 1997.
- [3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th International Conference on World Wide Web (WWW '01)*, ACM, 2001, pp. 285–295.
- [4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," in *Proc. 26th International Conference on World Wide Web (WWW '17)*, 2017, pp. 173–182.
- [7] R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society, Series B*, vol. 58, no. 1, pp. 267–288, 1996.
- [8] A. Hoerl and R. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [9] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.
- [10] GroupLens Research, "MovieLens 100K Dataset," University of Minnesota, 1998. [Online]. Available: <https://grouplens.org/datasets/movielens/100k/>
- [11] W. S. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conference (SciPy '10)*, 2010, pp. 51–56.
- [12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [13] Streamlit Inc., "Streamlit: A Faster Way to Build and Share Data Apps," 2019. [Online]. Available: <https://streamlit.io>
- [14] C. Aggarwal, "Recommender Systems: The Textbook," Springer International Publishing, 2016.
- [15] T. Hastie, R. Tibshirani, and J. Friedman, "The Elements of Statistical Learning," 2nd ed., Springer, 2009.

— End of Report —

Rakshit Thakur | 2023BTCSE013 | JLU07720 | B.Tech (CSE) | Machine Learning (STBT2102) | Jagran Lakecity University, Bhopal